College of Engineering Karunagappally

Engineering College Rd, Thodiyoor, Karunagappalli, Kerala 690523

Affiliated to APJ Abdul Kalam Technological University

Department of Computer Science and Engineering

Mini Project Report

PRISM AI BROWSER

An Intelligent Agentic Web Browser with Privacy-Centric AI Integration

Submitted by:

Nobin Sijo	(Roll No: KNP23CS082)
Prasanth P	(Roll No: KNP23CS086)
Sankar Narayan S	(Roll No: KNP23CS094)
Pranav P	(Roll No: KNP23CS085)

Under the Guidance of:

Ms. Sandhya S Krishnan

Assistant Professor, Dept. of CSE

(Project Co-ordinator)

Ms. Jilu Mariyam Jose

Assistant Professor, Dept. of CSE

(Project Guide)

Academic Year 2025–2026

COLLEGE OF ENGINEERING KARUNAGAPPALLY

Thodiyoor P.O. Karunagappally

Kollam, Kerala 690523

2025–26

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CERTIFICATE

This is to certify that the report entitled **Prism AI Browser** submitted by Nobin Sijo (KNP23CS082), Sankar Narayan S (KNP23CS094), Prasanth P (KNP23CS086), Pranav P (KNP23CS085) to the APJ Abdul Kalam Technological University in partial fulfillment of the B.Tech. degree in Computer Science and Engineering is a bonafide record of the project work carried out by them under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Project Co-ordinator

Ms. Sandhya S Krishnan

Asst. Professor, Dept. of CSE

Project Guide

Ms. Jilu Mariyam Jose

Asst. Professor, Dept. of CSE

Head Of The Department

Dr. SMITHA P

Dept. of CSE

DECLARATION

We hereby declare that the project report **Prism AI Browser**, submitted for partial fulfillment of the requirements for the award of degree of Bachelor of Technology of the APJ Abdul Kalam Technological University, Kerala is a bonafide work done by us under supervision of Ms. Sandhya Akhil. This submission represents our ideas in our own words and where ideas or words of others have been included, we have adequately and accurately cited and referenced the original sources. We also declare that we have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma or similar title of any other University.

Kollam

15-03-2026

Nobin Sijo

Sankar Narayan S

Prasanth P

Pranav P

Acknowledgement

We take this opportunity to express our deepest sense of gratitude and sincere thanks to everyone who helped us to complete this work successfully. We express our sincere thanks to the Head of Department, Computer Science and Engineering, College of Engineering Karunagappally for providing us with all the necessary facilities and support. We would like to express our sincere gratitude to Ms. Sandhya S Krishnan, Assistant Professor in CSE, College of Engineering Karunagappally, our Project Co-ordinator, for the support and co-operation. We would also like to place on record our sincere gratitude to our project guide Ms. Jilu Mariyam Jose, Assistant Professor in CSE, College of Engineering Karunagappally for the guidance and mentorship throughout this work. Finally we thank our families and friends who contributed to the successful fulfilment of this project work.

Nobin Sijo

Sankar Narayan S

Prasanth P

Pranav P

Abstract

Modern web browsers primarily act as passive tools for accessing information, offering limited intelligent assistance while browsing. With the growing demand for productivity, accessibility, and privacy-focused AI systems, there is a need for a browser that can actively assist users without relying heavily on cloud-based services.

This project presents **Prism AI Browser**, an AI-powered agentic, privacy-centric web browser built on a Chromium-based engine that enables intelligent, voice- and prompt-controlled interaction with web content. Prism integrates large language models (LLMs) to perform tasks such as website navigation, content understanding, browser automation, and system-level assistance.

The system employs agentic AI capabilities to interpret natural language commands and execute multi-step actions within the browser environment, enhancing user efficiency and accessibility. Key features include Agentic AI execution, voice-to-text control, intelligent tab and settings management, and extensibility through modular browser extensions.

Overall, Prism AI Browser proposes an intelligent browsing framework in which AI is integrated directly into the browser environment to support user interaction, automation, and accessibility. The project demonstrates the feasibility of combining agentic AI capabilities with a modern browser architecture to deliver a secure, efficient, and user-centric web experience.

Keywords: Agentic AI, Privacy-Centric Browser, Large Language Models, Voice Control, Chromium, Browser Automation.

Document Control Data Sheet

Table 1: Project Identification

Project Title	Prism AI Browser: An Intelligent Agentic Web Browser with Privacy-Centric AI Integration
Project Type	Mini Project B.Tech Project
Department	Computer Science and Engineering
Institution	College of Engineering Karunagappally
Academic Year	2025–2026
Report Version	1.0
Date	March 2026

Table 2: Team Members

S.No	**Name**	**Role**
1	Nobin Sijo	Project Lead, AI Backend Integration,Extension Development
2	Sankar Narayan S	Chromium Customisation, Toolbar UI
3	Prasanth P	Build System, Patch Management,Version Control
4	Pranav P	Voice Interface,Browser UI

Table 3: Document Revision History

Version	**Date**	**Author**	**Description**
0.1	Jan 2026	Nobin Sijo	Initial draft
0.5	Feb 2026	Sankar Narayan S	UML diagrams and system design added
1.0	Mar 2026	All Members	Final report complete

Table 4: Technology Stack

Component	**Technology / Tool**
Browser Engine	Chromium (open-source base)
AI Agent Backend	Python-based LLM agent server (port 7788)
Browser Automation Protocol	Chrome DevTools Protocol (CDP, port 9222)
UI Toolkit	Chromium Views (C++)
Build System	GN + Ninja
Voice Input	Web Speech API / local STT engine
Large Language Model	Local LLM (privacy-centric, on-device)
Patch Management	Git diff / format-patch

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Purpose

This document presents the final project report for *Prism AI Browser*, an intelligent, agentic, and privacy-centric web browser developed as part of the B.Tech mini project at the Department of Computer Science and Engineering, College of Engineering Karunagappally. The report describes the problem domain, the design and architecture of the system, its features, non-functional requirements, and UML-based system models.

1.2 Project Overview

Modern web browsers function primarily as passive navigation tools, offering limited intelligent assistance to users during browsing sessions. With growing demand for productivity-enhancing and privacy-respecting software, there is a clear need for a browser capable of actively understanding user intent and automating complex web interactions without relying on cloud-based AI services.

Prism AI Browser addresses this gap by integrating large language model (LLM) based agentic AI directly into a Chromium-based browser environment. Users can interact with the browser using natural language commands and voice input to perform tasks such as navigating websites, managing tabs, summarising content, filling forms, and executing multi-step automation sequences entirely on-device.

1.3 Scope

The scope of the Prism AI Browser project encompasses:

- Modification and customisation of the open-source Chromium browser source code.
- Development of a custom `PrismAIButton` toolbar component that connects the browser UI to the local AI agent backend.
- Integration of the Chrome DevTools Protocol (CDP) to allow the AI agent to programmatically control browser tabs, the DOM, and navigation.
- A locally running AI agent server (Python-based) that interprets natural language instructions and translates them into browser actions via CDP.
- Voice-to-text input for hands-free browser interaction.

- Privacy-first design: all AI processing is performed locally with no user data transmitted to external servers.

1.4 Definitions, Acronyms, and Abbreviations

Table 1.1: Definitions, Acronyms, and Abbreviations

Term	Definition
AI	Artificial Intelligence
LLM	Large Language Model
CDP	Chrome DevTools Protocol – a remote debugging and automation protocol exposed by Chromium on port 9222
Agentic AI	An AI system capable of autonomously planning and executing multi-step tasks
NLP	Natural Language Processing
STT	Speech-to-Text
DOM	Document Object Model – the in-memory tree representation of a web page
GN	Generate Ninja – the meta-build system used by Chromium
Ninja	A fast build system used alongside GN for Chromium compilation
API	Application Programming Interface
UI	User Interface
CSE	Computer Science and Engineering

1.5 References

- Chromium source code – https://chromium.googlesource.com/chromium/src
- Chrome DevTools Protocol – https://chromedevtools.github.io/devtools-protocol/
- OpenAI GPT-3 API documentation – https://beta.openai.com/docs/
- Local LLM frameworks (e.g., LLaMA, Alpaca) – https://github.com/ggerganov/llama.cpp
- Web Speech API documentation – https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API

1.6 Overview of the Report

The remainder of this report is organised as follows:

- **Chapter 2 – Overall Description:** Describes the product perspective, functions, user classes, assumptions, and dependencies.
- **Chapter 3 – Requirements Specification:** Details the functional and interface requirements of the system.
- **Chapter 4 – System Features:** Provides an in-depth description of each system feature.
- **Chapter 5 – Other Non-Functional Requirements:** Documents performance, security, usability, and other quality attributes.
- **Chapter 6 – UML Diagrams:** Presents Use Case, Class, Sequence, State Chart, and Flowchart diagrams.

Chapter 2

Overall Description

2.1 Product Perspective

Prism AI Browser is a new, self-contained product that is an evolution of the open-source Chromium browser. It does not replace Chromium; instead it extends Chromium's capabilities by:

- Adding a custom AI toolbar button (`PrismAIButton`) built in C++ using Chromium's Views UI framework.
- Guaranteeing that the Chrome DevTools Protocol (CDP) remote debugging server is always active on port 9222 from browser startup, enabling programmatic browser control.
- Shipping with a companion Python-based AI agent server that listens on `http://127.0.0.1:7788`, interprets natural language and voice commands, and drives the browser via CDP.

Figure 2.1 illustrates the high-level relationship between the Prism browser, the AI agent backend, and the user.

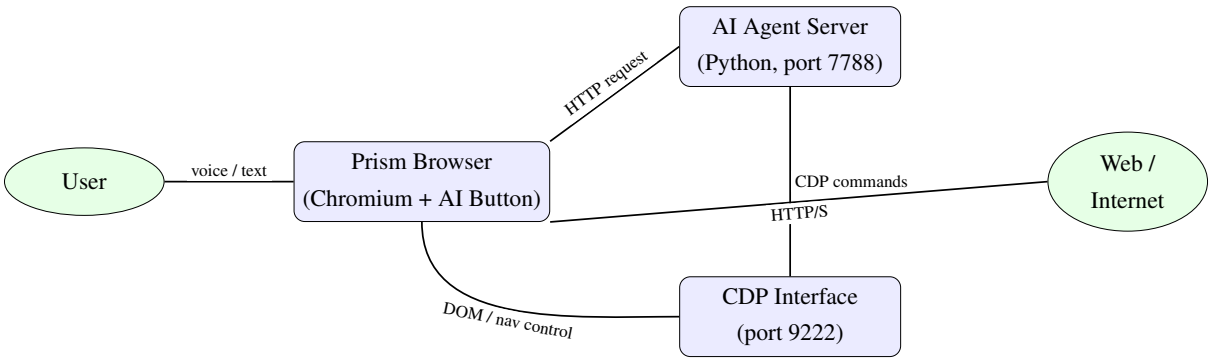

Figure 2.1: Prism AI Browser – System Context Diagram

2.2 Product Functions

The primary functions of Prism AI Browser are summarised below:

1. **Natural Language Browsing:** Accept text or voice-based natural language commands from the user and translate them into browser actions.
2. **Agentic Task Execution:** Break complex user requests into sub-tasks and execute them autonomously using multi-step reasoning.

3. **Tab Management:** Open, close, navigate, reload, and organise browser tabs in response to user instructions.
4. **Content Understanding:** Extract, summarise, and reason about web page content using the embedded LLM.
5. **Form Automation:** Automatically fill web forms with data provided or inferred by the AI agent.
6. **Voice Control:** Transcribe speech input using a local Speech-to-Text engine and pass the result to the AI agent.
7. **Settings Management:** Adjust browser settings (homepage, extensions, privacy controls) via natural language.
8. **Privacy-Centric Operation:** Process all AI inference locally; no user data or page content is sent to external servers.

2.3 User Classes and Characteristics

Table 2.1: User Classes and Characteristics

User Class	Description	Technical Level
General User	Everyday browser users wanting AI-assisted browsing and voice commands	Low – no technical knowledge required
Power User	Developers and researchers who use the AI agent for task automation and scripting	Medium – comfortable with CLI and settings
Accessibility User	Users with motor or visual impairments who rely on voice control	Low – interaction via speech only
Developer/Contributor	Engineers who extend, patch, and build Prism from source on Chromium	High – proficient with C++, GN, and git

2.4 Operating Environment

- **Host OS:** Linux (primary), macOS, and Windows 10 or later.
- **CPU:** x86-64 processor; ARM64 supported where Chromium supports it.

- **RAM:** Minimum 4 GB; 8 GB or more recommended for on-device LLM inference.
- **Storage:** Minimum 3 GB for the Chromium build; additional space for LLM model weights.
- **Network:** Standard TCP/IP; the AI agent communicates locally only (loopback interface).
- **Python:** Version 3.10 or later required for the AI agent server component.

2.5 Design and Implementation Constraints

- The project must remain a *patch set* on top of a specific Chromium base revision recorded in `BASE_REVISION.txt`, to facilitate reproducible builds.
- The CDP port (9222) is hardcoded at launch via a command-line switch injection in `chrome_main.cc` to guarantee availability.
- The AI agent backend URL (`http://127.0.0.1:7788`) is hardcoded in `prism_ai_button.cc`; it communicates only over the loopback interface (no internet exposure).
- All Chromium-specific branding strings (product name, descriptions) in `chromium_strings.grd` are replaced with "Prism" equivalents.
- The Prism icon (`prism_ai.icon`) is a 16×16 triangle vector icon registered in Chromium's vector icon build system.

2.6 Assumptions and Dependencies

- It is assumed that the user has a compatible machine capable of running the Chromium browser.
- The AI agent server must be started independently before using Prism AI features; it is not auto-started by the browser in the current version.
- The local LLM model weights must be downloaded and configured separately by the user or system administrator.
- `depot_tools` and a full Chromium source checkout are required to build Prism from source.

Chapter 3

Requirements Specification

3.1 External Interface Requirements

3.1.1 User Interfaces

- **Prism AI Toolbar Button:** A prism-shaped triangle icon button is embedded in the Chromium toolbar between the Home button and the tab strip. Clicking it opens the AI agent web interface at `http://127.0.0.1:7788` in a new foreground tab.
- **AI Agent Web UI:** A local web-based chat interface served by the Python backend. Users type natural language commands or activate voice input. The interface displays AI responses, progress indicators for multi-step tasks, and results.
- **Voice Input:** A microphone button within the AI agent interface activates Speech-to-Text (STT) capture. Recognised text is pre-filled into the command input field.
- **Browser Standard UI:** All standard Chromium browser UI elements (address bar, back/forward/reload navigation, bookmarks, extensions toolbar) remain intact.

3.1.2 Hardware Interfaces

- Microphone: For voice command input via the STT subsystem.
- Display: Standard monitor supporting at minimum 1024×768 resolution.
- Network Interface: Required for standard web browsing; AI agent operates on the loopback interface only.

3.1.3 Software Interfaces

- **Chrome DevTools Protocol (CDP):** The AI agent connects to the Chromium browser via WebSocket on port 9222. CDP provides APIs for target management (`Target.*`), page navigation (`Page.*`), DOM manipulation (`DOM.*`), input simulation (`Input.*`), and JavaScript execution (`Runtime.*`).
- **LLM Inference API:** The AI agent backend communicates with a local LLM (e.g., via Ollama or llama.cpp HTTP API) for language understanding and response generation.
- **Web Speech API / STT Engine:** Used for voice-to-text transcription, either via the browser's built-in Web Speech API or a locally hosted STT service.

3.1.4 Communication Interfaces

- HTTP on `127.0.0.1:7788`: Browser-to-agent communication for submitting commands.
- WebSocket on `127.0.0.1:9222`: Agent-to-browser CDP communication for automation.
- All external internet communication is solely for normal web browsing and uses standard HTTP/HTTPS.

3.2 Functional Requirements

3.2.1 FR-01: AI Button Integration

Table 3.1: FR-01: AI Toolbar Button Integration

Requirement ID	FR-01
Title	AI Toolbar Button
Description	The browser shall display a Prism AI toolbar button (prism triangle icon) in the main toolbar.
Input	User clicks the Prism AI button.
Output	A new foreground tab opens navigating to `http://127.0.0.1:7788`.
Priority	High

3.2.2 FR-02: CDP Always-On

Table 3.2: FR-02: Guaranteed CDP Availability

Requirement ID	FR-02
Title	Guaranteed CDP Availability
Description	The CDP remote debugging server shall always start on port 9222 at browser launch, even if no explicit command-line flag is provided.
Input	Browser process startup.
Output	CDP WebSocket endpoint active at `ws://127.0.0.1:9222`.
Priority	High

3.2.3 FR-03: Natural Language Command Processing

Table 3.3: FR-03: Natural Language Command Processing

Requirement ID	FR-03
Title	NLP Command Execution
Description	The AI agent shall accept a natural language command as text input, interpret it using an LLM, decompose it into sub-tasks, and execute each sub-task via CDP.
Input	Natural language command string (e.g., "Open Wikipedia and search for quantum computing").
Output	Browser performs the requested action(s); agent returns a status message.
Priority	High

3.2.4 FR-04: Voice Command Input

Table 3.4: FR-04: Voice Command Input

Requirement ID	FR-04
Title	Voice-to-Text Input
Description	The AI agent UI shall provide a voice input button; speech is transcribed to text and submitted as a command.
Input	User activates voice input and speaks a command.
Output	Transcribed text is populated in the command field and submitted to the agent.
Priority	Medium

3.2.5 FR-05: Tab Management

Table 3.5: FR-05: AI-Driven Tab Management

Requirement ID	FR-05
Title	AI-Driven Tab Management
Description	The AI agent shall be able to open, close, switch, and navigate browser tabs in response to user commands.
Input	Natural language command related to tab management.
Output	Browser tabs are created, destroyed, or switched as instructed.
Priority	High

3.2.6 FR-06: Page Content Extraction and Summarisation

Table 3.6: FR-06: Page Content Extraction and Summarisation

Requirement ID	FR-06
Title	Content Understanding
Description	The AI agent shall extract visible text from the active tab and pass it to the LLM to produce summaries, answer questions, or extract structured data.
Input	User command such as “Summarise this page” or “What is the price of this product?”
Output	AI-generated summary or answer displayed in the chat UI.
Priority	Medium

3.2.7 FR-07: Form Auto-Fill

Table 3.7: FR-07: Intelligent Form Auto-Fill

Requirement ID	FR-07
Title	Intelligent Form Automation
Description	The AI agent shall identify form fields in the current page’s DOM and fill them with data provided by or inferred from the user’s command.
Input	Command such as “Fill in my name and email in this form” with associated data.
Output	Form fields populated with correct data via CDP input simulation.
Priority	Medium

3.2.8 FR-08: Privacy-First Local Processing

Table 3.8: FR-08: Privacy-First Local Processing

Requirement ID	FR-08
Title	Local-Only AI Processing
Description	No user commands, page content, or browsing history shall be transmitted to any external server as part of the AI processing pipeline.
Input	Any AI feature usage.
Output	All LLM inference and STT processing occurs locally on the user's machine.
Priority	High

Chapter 4

System Features

4.1 Feature 1: Prism AI Toolbar Button

4.1.1 Description and Priority

The Prism AI Button is a custom toolbar component integrated into the Chromium browser toolbar. It serves as the primary entry point for all AI-assisted features. It is a **high-priority** feature as it ties the browser UI to the AI backend.

4.1.2 Implementation Details

The button is implemented in C++ as the class `PrismAIButton`, which extends Chromium's `ToolbarButton` class. Key implementation aspects:

- The class is defined in `chrome/browser/ui/views/toolbar/prism_ai_button.h` and implemented in `prism_ai_button.cc`.
- It uses the custom vector icon `kPrismAiIcon` defined in `chrome/app/vector_icons/prism_ai.icon`, which renders a prism-shaped double triangle on a 16×16 canvas.
- The tooltip text is set to "Prism AI".
- When clicked, `OnButtonPressed()` opens a new foreground tab to `http://127.0.0.1:7788` using the browser's `OpenURL()` method with `PAGE_TRANSITION_TYPED` transition type.
- The button is added as a child view in `ToolbarView::Init()` after the Home button, and a raw pointer `prism_ai_button_` is maintained in `ToolbarView`.

4.1.3 Functional Requirements Supported

FR-01

4.2 Feature 2: Always-On Chrome DevTools Protocol (CDP)

4.2.1 Description and Priority

CDP is the communication backbone between the AI agent and the browser. Without CDP being active, the AI agent cannot control or inspect browser state. This feature is **high-priority**.

4.2.2 Implementation Details

In `chrome/app/chrome_main.cc`, the following logic is injected at browser startup:

- After the command line is initialised, the code checks whether `--remote-debugging-port` or `--remote-debugging-pipe` switches are already set.
- If neither is present, `--remote-debugging-port=9222` is appended programmatically, guaranteeing CDP is always available.
- This eliminates the need for users to start the browser with special flags.

The CDP endpoint is accessible at `ws://127.0.0.1:9222` from the local machine only, providing a WebSocket interface for tab management, DOM access, JavaScript execution, and input simulation.

4.2.3 Functional Requirements Supported

FR-02

4.3 Feature 3: AI Agent Backend

4.3.1 Description and Priority

The AI agent server is the intelligence core of Prism. It is a Python application that serves the chat UI, processes user commands using an LLM, and drives the browser via CDP. This is a **high-priority** feature.

4.3.2 Implementation Details

- The server listens on `http://127.0.0.1:7788` (loopback only).
- On receiving a user command, the server sends the command to a locally-running LLM (e.g., via the Ollama REST API at `http://127.0.0.1:11434`).
- The LLM decomposes the command into a sequence of browser automation actions (a "plan").
- Each action is executed by sending the corresponding CDP command over WebSocket to the browser at `ws://127.0.0.1:9222`.
- Common CDP actions include: `Target.createTarget`, `Page.navigate`, `DOM.getDocument`, `Input.dispatchMouseEvent`, and `Runtime.evaluate`.
- Results and status messages are returned to the web UI in real time.

4.3.3 Functional Requirements Supported

FR-03, FR-05, FR-06, FR-07, FR-08

4.4 Feature 4: Voice-to-Text Input

4.4.1 Description and Priority

Voice command support makes Prism AI Browser accessible to users with limited mobility and enables hands-free productivity. This is a **medium-priority** feature.

4.4.2 Implementation Details

- The AI agent web UI provides a microphone button that activates the browser's Web Speech API or a locally hosted STT engine.
- The transcribed speech is populated in the text command field.
- The user can review the transcribed text before submitting or it can be submitted automatically.
- All audio processing is performed locally to maintain privacy compliance.

4.4.3 Functional Requirements Supported

FR-04

4.5 Feature 5: Content Summarisation and Q&A

4.5.1 Description and Priority

Prism AI Browser can extract and summarise web page content, enabling users to quickly understand long articles, technical documents, or e-commerce product pages. This is a **medium-priority** feature.

4.5.2 Implementation Details

- The AI agent executes `document.body.innerText` via `Runtime.evaluate` CDP call to extract page text.
- The extracted text and user query are assembled into an LLM prompt.
- The LLM generates a concise summary or direct answer.
- Results are streamed back to the chat UI.

4.5.3 Functional Requirements Supported

FR-06

4.6 Feature 6: Chromium Rebranding to Prism

4.6.1 Description and Priority

All user-visible strings in the browser referring to "Chromium" are replaced with "Prism" to establish a distinct brand identity. This is a **low-priority** (cosmetic) feature but essential for the final product.

4.6.2 Implementation Details

- Modifications are made to `chrome/app/chromium_strings.grd`, the central string resource file for Chromium.
- Over 100 user-facing strings are updated including: product name, window titles, installer messages, error messages, and taskbar/dock pin prompts.
- The short product name becomes "Prism"; the full description reads: *"Prism is an AI agentic powered web browser that runs webpages and applications with lightning speed."*.

4.6.3 Functional Requirements Supported

FR-01 (brand identity)

4.7 Feature 8: Prism Homepage DevSpace

4.7.1 Description and Priority

The Prism Homepage DevSpace is a custom, developer-oriented new-tab homepage served locally by the Prism browser. It replaces the standard blank new-tab page with a productivity-focused dashboard. This is a **medium-priority** feature that significantly enhances the day-to-day developer experience.

4.7.2 Overview

On opening a new tab, users are greeted with a full-screen homepage (`index.html`) featuring a live clock, motivational quotes, animated particle and matrix backgrounds, and a scrollable Dev Space panel. The homepage is designed specifically for the Prism AI Browser but also works in any modern browser when served via a local HTTP server (Python or Node.js).

4.7.3 Key Sub-Features

Live Clock and Greetings

- A large, prominent time display (`#zen-time`) supports both 12-hour and 24-hour formats selectable from settings.

- Personalised motivational greetings are shown as primary and secondary quotes sourced from `quotes.json` and `dynamic_quotes.json`.
- The greeting updates daily, creating a fresh experience on each browser session.

Dev Space Panel

Scrolling below the clock reveals the Dev Space — a card-based launcher grid offering quick access to developer tools and utilities:

Table 4.1: DevSpace Developer Tools

Tool Card	**Description**
Localhost	Quick-launch link to `http://localhost:3000` (configurable) for local web dev servers
GitHub	Direct link to `github.com` for version control and repository management
Dev Console	Opens an interactive in-page developer tools panel with browser shortcuts
MDN Docs	Link to Mozilla Developer Network for web API documentation
Color Generator	Opens `dev-space/color-gen.html` — an interactive colour palette generator
Prompt Synthesizer	Opens `dev-space/prompt-synthesizer.html` — an AI-powered prompt enhancement and refinement tool
Speed Test	Embedded network speed test via `dev-space/speedtest/`
Notepad Panel	Floating persistent notepad (`dev-space/notepad-panel.html`) that saves notes to `localStorage`
Focus Settings	Configurable focus/distraction-free mode from `dev-space/focus-settings.html`
Matrix Tester	Visual demo and configurator for the animated Matrix rain background effect

Animated Backgrounds

- `matrix.js` renders a customisable Matrix-style green rain effect on a canvas element.
- Floating particle animations (`createParticle()`) add ambient depth to the background.
- Users can switch between wallpapers from the `images/Wallpapers/` collection.

System Information Display

- RAM usage is displayed in real time using the browser's `performance.memory` API (where available).
- Clock style selection is persisted via `localStorage`, with previews available in the settings panel (`clock-previews/`).

4.7.4 Implementation Details

- Built with plain HTML, CSS (`style.css`, Space Grotesk / JetBrains Mono fonts), and vanilla JavaScript (`script.js`).
- No build step required for the homepage itself; it is served as static files.
- Must be served from a local web server (not `file://`) due to CORS restrictions on `quotes.json` fetch calls.
- Integrates tightly with Prism by linking to internal browser pages via special protocols, with fallback to standard URLs for compatibility.

4.7.5 Functional Requirements Supported

Extended browser UX, developer productivity

4.8 Feature 9: Aura AI Chat (with Image Generation)

4.8.1 Description and Priority

Aura AI Chat is the primary conversational AI interface integrated into Prism Browser, accessible via a dedicated local web page. It supports multi-model text chat, AI-powered image generation, agentic task execution (browser automation), and persistent conversation history management. This is a **high-priority** feature representing the core user-facing AI capability of Prism.

4.8.2 Overview

The chat interface (`index.html`, branded "Aura – Your Agentic AI Assistant") is built as a single-page application with a collapsible sidebar for conversation history, a main chat area, and a bottom input bar supporting both text and voice entry. It uses the `#88E755` (Prism green) primary colour scheme and the *Space Grotesk* / *JetBrains Mono* font stack with a dark `#0a0a0a` background.

4.8.3 AI Models Supported

Table 4.2: AI Models Supported by Aura Chat

Mode	Model	Description
Aura Mode	Gemini Pro (Google)	Default chat model via Gemini API; best for general conversation, reasoning, and summarisation
DeepSeek V3.2	DeepSeek-V3.2-Exp (Hugging Face)	Advanced reasoning model; first request may take 20–30 s while the model loads on the Hugging Face inference server

Users switch between models via the "Select AI Service" dropdown in the interface.

4.8.4 AI Image Generation

- **Engine:** FLUX.2 Klein (`black-forest-labs/flux.2-klein-4b`) via the OpenRouter unified AI gateway API.
- **Trigger:** Users activate image generation by natural language (e.g., "generate image of a futuristic city", "draw a cute robot").
- **Display:** Generated images are returned as Base64-encoded inline data URIs and rendered directly in the chat. A direct URL fallback is provided.
- **Caching:** Image results are cached to avoid redundant API calls for repeated prompts.
- **Download:** Users can download generated images directly from the chat message.

4.8.5 Agentic Task Execution

The `task-executor.js` module, instantiated as a `TaskExecutor` class, intercepts commands with recognised patterns using regular expressions and executes real browser actions. Supported task categories:

Table 4.3: Supported Agentic Task Execution Categories

Category	Example Commands
YouTube	"open youtube and search cocomelon"
Search Engines	"google search artificial intelligence", "search wikipedia quantum physics"
Social Media	"open twitter and search AI news", "open reddit and search programming", "open github and search react"
Entertainment	"play imagine dragons on spotify", "open netflix and search stranger things", "search amazon wireless headphones"

Table 4.3: Supported Agentic Task Execution Categories

Category	Example Commands
Productivity	"set timer for 5 minutes", "create event for team meeting tomorrow", "check weather in New York", "open calculator"
Navigation	"open maps coffee shops near me", "send email to user@example.com"
General Web	"open github.com", "open google.com"

All tasks that open external URLs or perform browser navigation require user confirmation via an in-UI dialog. A "Don't show again" option is available for frequently approved actions.

4.8.6 Conversation History Management

- Conversations are stored persistently in `localStorage` under named sessions, listed in the collapsible left sidebar.
- A search bar (`#history-search`) filters conversations by keyword in real time.
- Export and import of chat history as JSON files is supported via the sidebar footer buttons.
- "New Chat" creates a fresh session; "Clear History" removes all saved sessions.

4.8.7 Memory Integration

Two optional memory backends are supported:

- `local-memory.js`: Stores user context (facts the AI learns about the user) in `localStorage` for offline, privacy-first persistent memory.
- `supermemory.js`: Optional integration with the Supermemory cloud service for cross-device memory synchronisation (requires API key; opt-in only to preserve privacy goals).

4.8.8 Error Handling and Reliability

- **Gemini API 503:** Exponential backoff retry with up to 5 attempts (delays: 1 s, 2 s, 4 s, 8 s, 16 s). Real-time retry status is shown to the user.
- **DeepSeek 503 (model loading):** Same exponential backoff strategy; user is informed that model loading may take 20–30 s.
- **401/403:** Authentication errors with clear user guidance on API key setup.
- **429:** Rate limit detection with a wait-and-retry message.
- **Network errors:** Connection issue detection with actionable error messages.

4.8.9 Implementation Details

- Built with HTML5, Tailwind CSS (via CLI build – `input.css` → `styles.css`), and vanilla JavaScript.
- Tailwind custom colour: `primary: #88E755` (Prism green); config in `tailwind.config.js`.
- Lucide icons loaded from jsDelivr CDN (switched from unpkg to fix MIME type error).
- Production CSS built via `npm run build:css:prod` (minified); development via `npm run build:css` (watch mode).

4.8.10 Functional Requirements Supported

FR-03 (NLP command processing), FR-04 (voice input), FR-06 (content summarisation), FR-08 (privacy-first local processing)

4.9 Feature 7: Patch-Based Build and Distribution System

4.9.1 Description and Priority

Prism maintains all its customisations as a compact set of Git patches applied on top of a pinned Chromium base revision. This allows contributors to reproduce the build on any machine without hosting the full Chromium repository. This is a **high-priority** operational feature.

4.9.2 Implementation Details

- `BASE_REVISION.txt`: Records the Chromium commit SHA that Prism patches are based on.
- `patches/0001-prism-working-tree.diff`: Contains all modifications to existing Chromium files (diff format).
- `patches/0002-prism-untracked-new-files.diff`: Contains new files added by Prism (vector icon, AI button source files).
- `scripts/apply_patches.sh`: Validates the Chromium checkout revision and applies all patches in order. Supports options `--check`, `--skip-base-check`, and `--patch-dir`.
- `scripts/build_prism.sh`: One-command wrapper that applies patches and invokes `gn gen` + `ninja` to build the `chrome` target.

4.9.3 Functional Requirements Supported

The patch-based build and distribution system is a foundational infrastructure component that underpins every functional requirement in this document. Without a reproducible, patch-managed build pipeline, none of the Prism-specific features could be reliably compiled, tested, or delivered to end users. Specifically:

- **FR-01 – FR-08 (all features):** Every Prism capability – the AI toolbar button, CDP integration, AI agent server, voice processing, tab management, form automation, content summarisation, and privacy-first inference – is delivered exclusively through the patch set applied on top of Chromium. A broken or non-reproducible patch application would prevent any of these features from functioning.
- **Reproducible Builds:** `BASE_REVISION.txt` pins the exact Chromium commit SHA, ensuring that the C++ patches in `patches/0001-*` and `patches/0002-*` always apply cleanly against the intended source tree, regardless of upstream Chromium development activity.
- **Complete Runtime Deployment:** The packaging stage (Stage 6) bundles the compiled Prism binary together with the AI agent (`ai-chat/`) and the custom new-tab homepage (`homepage-devspace/`) into a single distributable archive. This ensures all runtime components required by FR-01 through FR-08 are co-deployed and version-matched.
- **Patch Drift Detection:** If an upstream Chromium change modifies a file touched by a Prism patch, `apply_patches.sh` fails immediately with a conflict report, protecting the integrity of all features that depend on those patched files.

Chapter 5

Other Non-Functional Requirements

5.1 Performance Requirements

- **Browser Startup Overhead:** The CDP switch injection at startup adds negligible overhead (single conditional check on the command-line object) with no measurable impact on browser launch time.
- **AI Command Latency:** Simple single-step commands (e.g., "open a new tab") shall execute within 2 seconds on a machine with a modern LLM running locally. Multi-step agentic tasks may take longer depending on the complexity of the plan and network page load times.
- **LLM Inference Speed:** For smooth interaction, token generation by the local LLM shall sustain at least 10 tokens/second on recommended hardware (8 GB RAM, modern CPU).
- **CDP Round-Trip Latency:** CDP commands sent over the loopback WebSocket shall complete with latency under 10 ms, excluding page-load wait time.
- **UI Responsiveness:** The browser UI (toolbar, tabs, address bar) shall remain fully responsive during AI agent processing, as the agent runs in a separate Python process.

5.2 Safety Requirements

- The AI agent shall not execute browser actions that could cause irreversible data loss (e.g., deleting browser profiles) without explicit user confirmation.
- The CDP interface is bound to the loopback address (`127.0.0.1`) only; it must not be exposed on any network interface accessible from the internet.
- AI-generated JavaScript executed via `Runtime.evaluate` shall be sandboxed within the browser's existing renderer security model.

5.3 Security Requirements

- **No Data Exfiltration:** The AI agent shall establish no connections to external servers for AI processing. All LLM inference and STT processing shall be performed on-device.
- **Loopback Binding:** Both the AI agent server (port 7788) and the CDP endpoint (port 9222) shall accept connections from `127.0.0.1` only.

- **Session Isolation:** The AI agent shall not access content from other browser profiles or user sessions without explicit permission.
- **Input Sanitisation:** User commands passed to the LLM shall be sanitised to prevent prompt injection attacks that could cause the model to generate harmful CDP instructions.
- **Chromium Security Baseline:** All security features inherited from Chromium (sandboxed renderer processes, site isolation, safe browsing) shall remain fully active in Prism.

5.4 Software Quality Attributes

5.4.1 Reliability

- The browser shall be at least as stable as the underlying Chromium base revision on which it is built.
- If the AI agent server is not running, clicking the Prism AI button shall open a new tab showing a connection error page; the rest of the browser shall function normally.
- If a CDP command fails or times out, the AI agent shall report an informative error to the UI and allow the user to retry.

5.4.2 Maintainability

- All Prism-specific code shall be clearly separated from upstream Chromium code through focused, well-described patch files.
- Patch files shall be regenerated against a new Chromium base revision when the upstream drifts significantly, using the procedure documented in the `README.md`.
- New features shall each be captured in separate, logically named patch files (e.g., `0003-voice-input.diff`) to enable independent review and rollback.

5.4.3 Usability

- The Prism AI Button shall be immediately identifiable in the toolbar with a distinct icon and a clear "Prism AI" tooltip on hover.
- The AI agent chat interface shall provide clear feedback: a progress indicator during multi-step task execution, and colour-coded success/failure messages on completion.
- Voice commands shall be acknowledged with a visual recording indicator in the UI.
- Users with no technical knowledge shall be able to perform basic AI-assisted browsing tasks without consulting documentation.

5.4.4 Portability

- Prism shall build and run on all platforms supported by Chromium: Linux (primary), macOS, and Windows 10 or later.
- The patch set shall be platform-independent where possible; platform-specific code sections in the patches shall use Chromium's `BUILDFLAG(IS_WIN)`, `BUILDFLAG(IS_MAC)`, and `BUILDFLAG(IS_LINUX)` guards.

5.4.5 Extensibility

- The AI agent backend shall be designed with a modular tool interface, allowing new browser automation capabilities (e.g., screenshot capture, PDF export) to be added as separate tool modules without modifying core agent logic.
- Chromium extensions that interact with the Prism AI agent via the standard Chrome Extension Messaging API shall be supportable.

5.5 Business Rules

- Prism AI Browser is developed for educational and research purposes as a mini project.
- The project is open-source and its patch repository is publicly available on GitHub.
- The browser may not be distributed commercially without compliance with the Chromium / BSD licence terms.

Chapter 6

UML Diagrams

6.1 Use Case Diagram

The Use Case Diagram in Figure 6.1 identifies all actors and the principal use cases of the Prism AI Browser system.

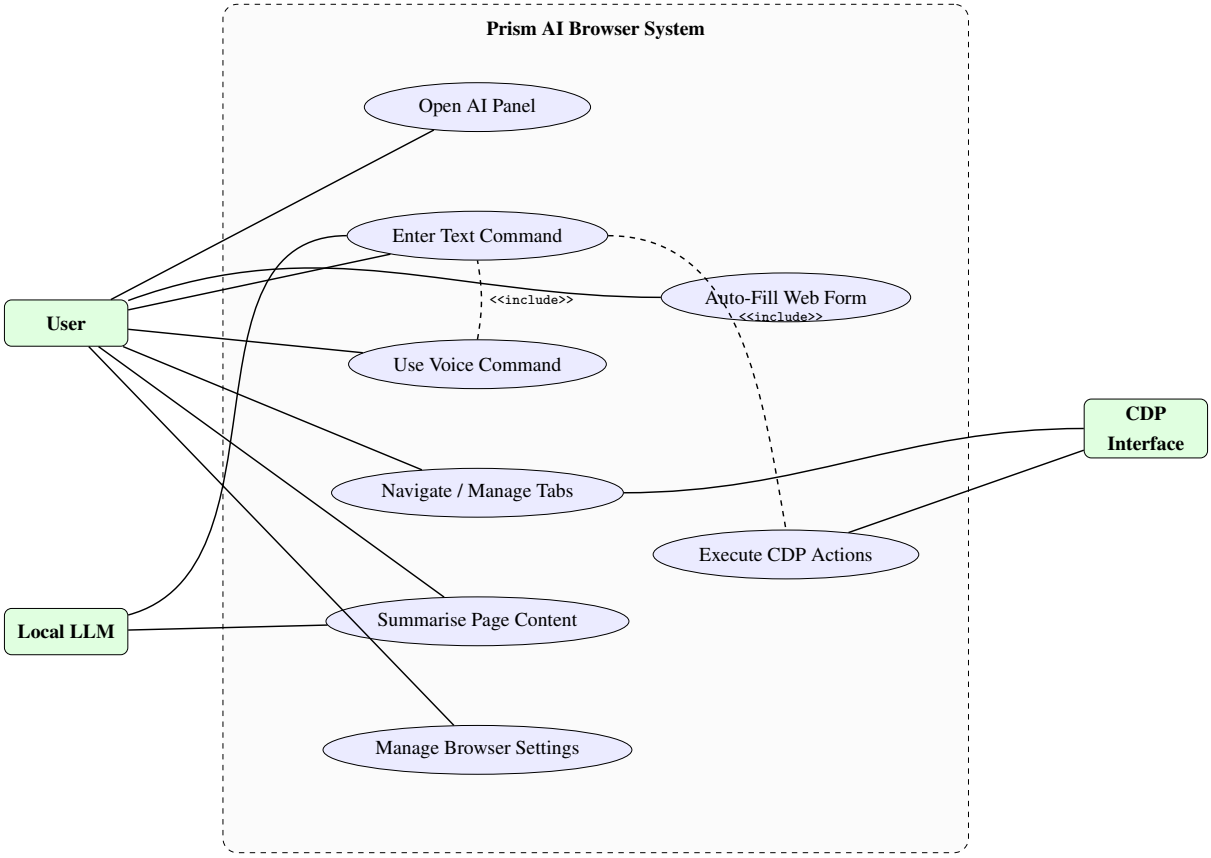

Figure 6.1: Use Case Diagram – Prism AI Browser

6.2 Class Diagram

Figure 6.2 shows the principal classes and their relationships in the Prism AI Browser system.

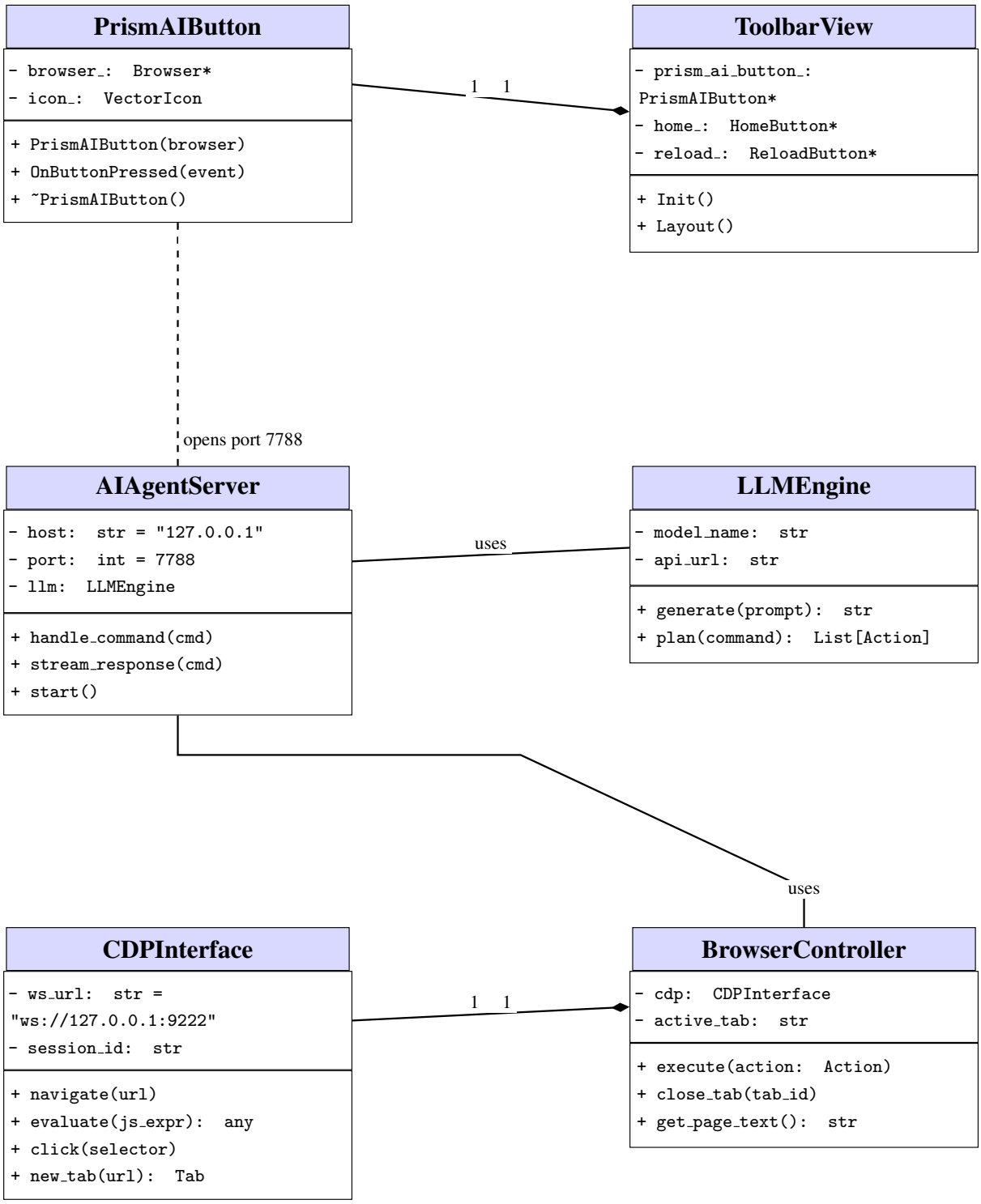

Figure 6.2: Class Diagram – Prism AI Browser

6.3 Sequence Diagram

Figure 6.3 illustrates the message flow for a typical natural language command: *"Open Wikipedia and search for quantum computing"*.

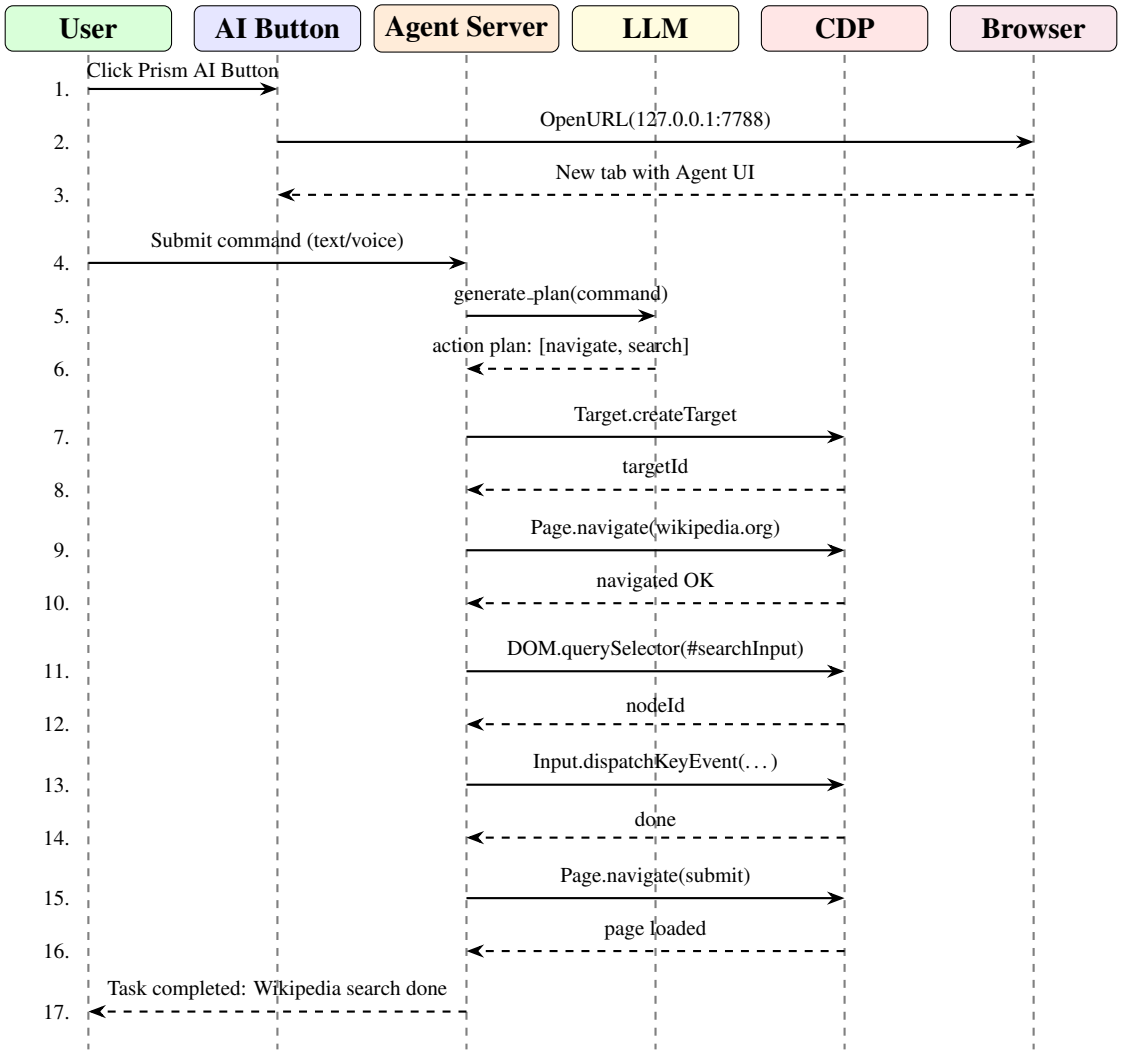

Figure 6.3: Sequence Diagram – Natural Language Command Execution

6.4 State Chart Diagram

Figure 6.4 shows the lifecycle of the AI Agent for a single user command, from idle to completion or error recovery.

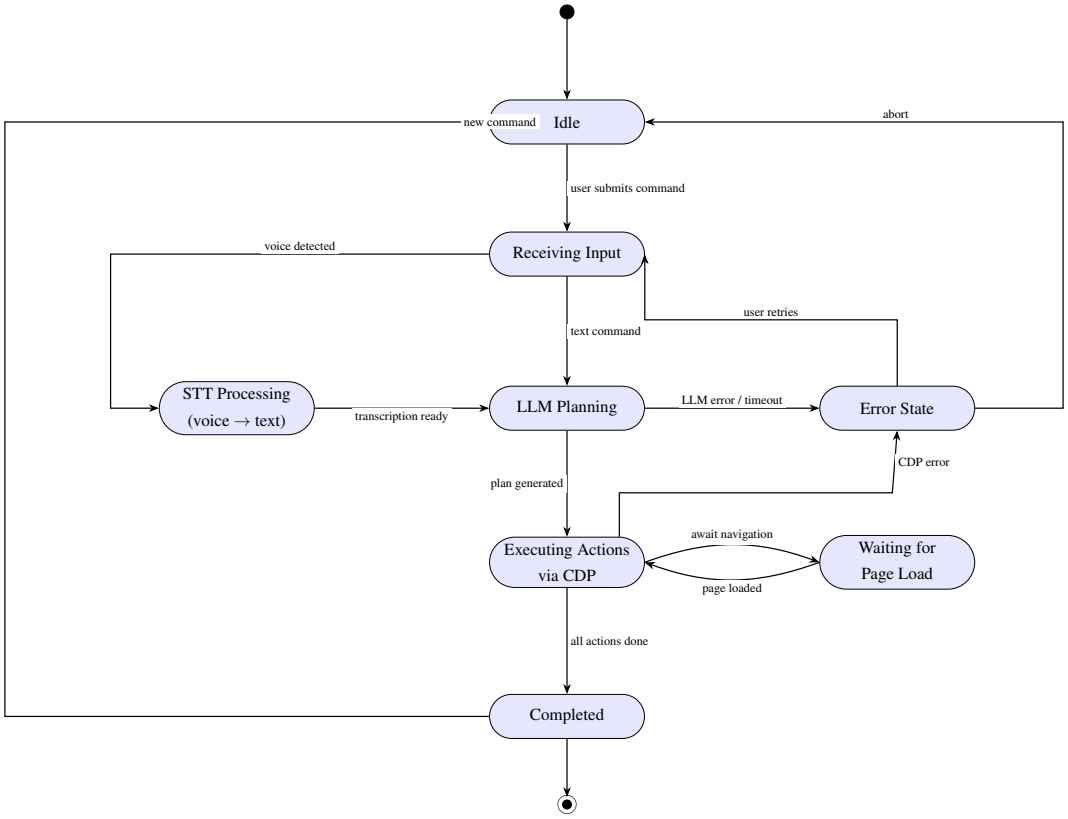

Figure 6.4: State Chart Diagram – AI Agent Command Lifecycle

6.5 Flowchart Diagram

Figure 6.5 presents the end-to-end processing flow for a user command in Prism AI Browser (voice or text).

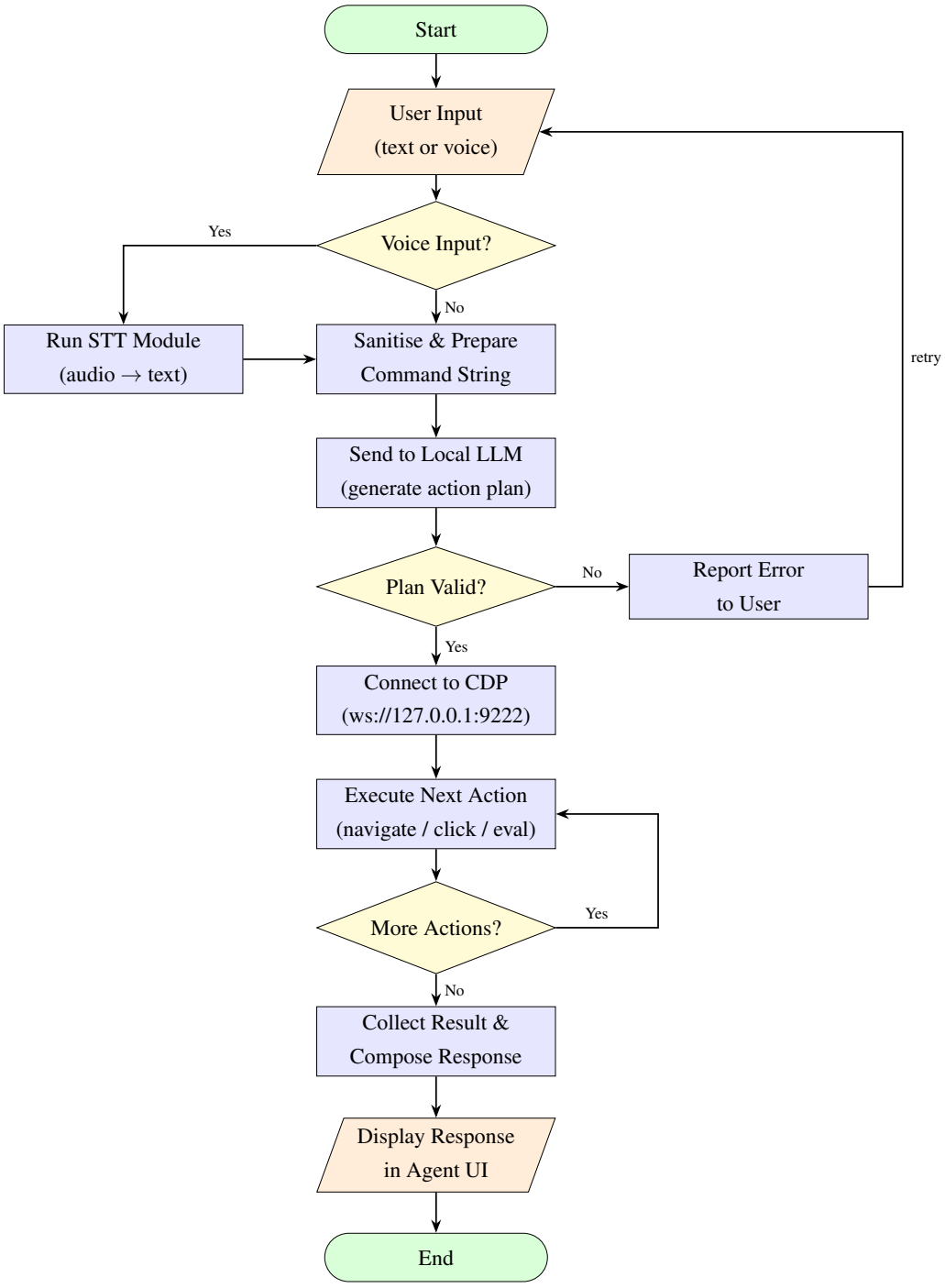

Figure 6.5: Flowchart – User Command Processing in Prism AI Browser

Chapter 7

Testing and CI/CD

7.1 Overview of Testing Strategy

Prism AI Browser is tested at multiple levels corresponding to its layered architecture: the Chromium browser layer (C++), the AI agent backend (Python), the chat/homepage frontend (JavaScript), and the end-to-end integration of all components via CDP. The testing strategy combines manual exploratory testing during development with automated checks embedded in the build pipeline.

7.2 Unit Testing

7.2.1 Browser Layer (C++)

Chromium's own GTest-based testing framework is used for unit testing the Prism-specific C++ components:

- **PrismAIButton:** Tests verify that `OnButtonPressed()` constructs the correct `OpenURLParams` (URL `http://127.0.0.1:7788`, disposition `NEW_FOREGROUND_TAB`, transition `PAGE_TRANSITION_TYPED`).
- **CDP Switch Injection:** A unit test confirms that launching the browser without `--remote-debugging-port` results in the switch being appended with value `9222`, and that an existing value is not overwritten.
- **Toolbar Integration:** `ToolbarView` tests assert that the `prism_ai_button_` child view is non-null after `Init()` and positioned in the correct order relative to the Home button.

7.2.2 AI Agent Backend (Python)

Python's built-in `unittest` / `pytest` framework is used to test the AI agent server modules:

- **LLMEngine:** Mock tests verify that prompts are correctly formatted and that the action plan parser produces a well-formed list of `Action` objects from LLM output.
- **CDPInterface:** Stubbed WebSocket connections test that `navigate()`, `evaluate()`, and `click()` methods produce correctly structured CDP message payloads.
- **BrowserController:** Tests verify that the action dispatcher routes each action type to the correct CDP method and handles errors (timeouts, invalid selectors) gracefully.

7.2.3 Aura AI Chat Frontend (JavaScript)

- **TaskExecutor:** Each regex task pattern is tested in isolation against a battery of natural language command variants to confirm correct handler dispatch (e.g., "open youtube and look for cocomelon" maps to the YouTube search handler).
- **Exponential Backoff Retry:** Unit tests mock the fetch API to return 503 responses and verify that the retry logic fires exactly 5 times with the expected delays (1 s, 2 s, 4 s, 8 s, 16 s) before surfacing an error.
- **Conversation Storage:** Tests confirm that loading, saving, importing, and exporting conversation sessions from `localStorage` round-trips correctly.
- **Image Generation:** Mocked OpenRouter API responses are tested to confirm Base64 images are rendered inline and the fallback URL path is triggered when Base64 data is absent.

7.3 Integration Testing

7.3.1 Browser-to-Agent Integration

- The integration test suite starts both the compiled Prism browser and the Python AI agent server, then drives actions through the CDP WebSocket to confirm end-to-end command execution.
- A test script submits the command "Navigate to example.com" and asserts that the active tab's URL changes to `https://example.com` within a 5-second timeout.
- Tab creation, closure, and switching are similarly exercised end-to-end.

7.3.2 CDP Availability Test

A lightweight Python script connects to `ws://127.0.0.1:9222/json` immediately after browser startup and asserts that a valid JSON list of targets is returned, confirming the CDP always-on feature works correctly.

7.3.3 Homepage DevSpace Integration

The homepage is tested in a locally served environment to confirm:

- `quotes.json` is fetched without CORS errors at `http://localhost:8000`.
- Clock updates in real time and persists the selected format across page reloads.
- All Dev Space tool cards open their respective overlays or links correctly.
- The Matrix background (`matrix.js`) renders without JavaScript errors across Chromium and Firefox.

7.4 System and Acceptance Testing

- **Manual Exploratory Testing:** Each release candidate is tested by team members across Windows and Linux for all major features: AI button, voice commands, image generation, task executor, and homepage.
- **Cross-Platform Build Verify:** The fully patched build is verified to compile without errors on Ubuntu 22.04 (primary CI target) and Windows 11.
- **Privacy Audit:** Network traffic is inspected with `mitmproxy` during AI feature usage to confirm that no user data or page content is sent to external servers. Only OpenRouter API calls (for image generation) and Gemini/HuggingFace calls (for chat) use the internet, and these are user-initiated with explicit API key configuration.
- **Performance Benchmarking:** Browser startup time is compared against the unpatched Chromium base revision to confirm the CDP switch injection adds $<$50 ms overhead.

7.5 CI/CD Pipeline for Prism Browser

7.5.1 Pipeline Architecture

The Prism CI/CD pipeline is implemented as a GitHub Actions workflow that validates, builds, and optionally releases a Prism build on every push to the main branch. Figure 7.1 shows the pipeline stages.

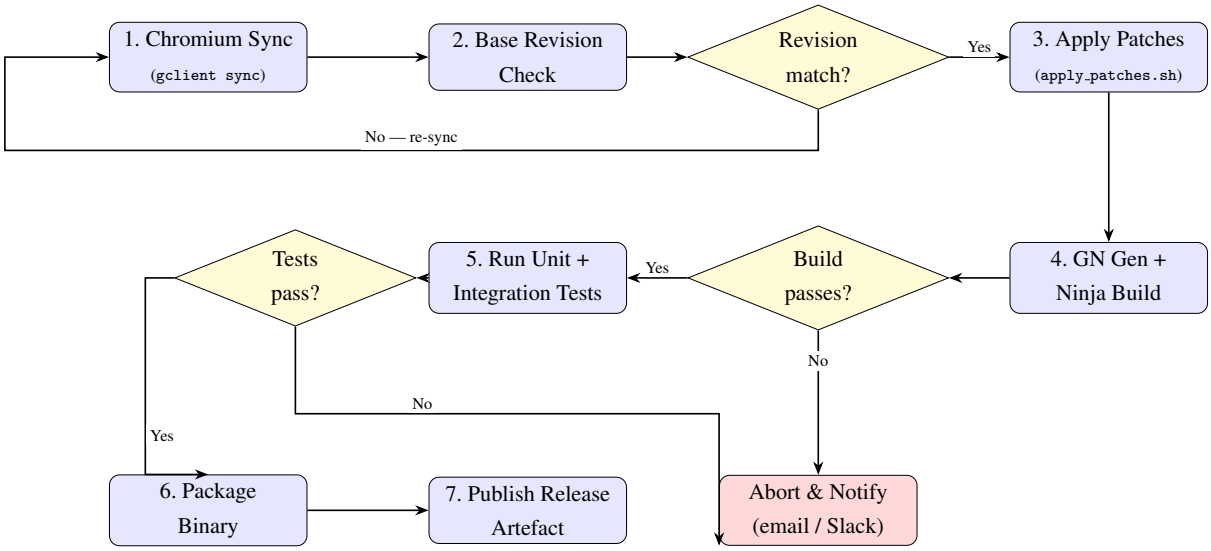

Figure 7.1: CI/CD Pipeline – Prism AI Browser

7.5.2 Stage Details

Table 7.1: CI/CD Pipeline Stage Details

Stage	Tool / Script	Description
1	`gclient sync` / `fetch chromium`	Syncs the Chromium source tree to the SHA recorded in `BASE_REVISION.txt`. Skippable with `--skip-sync` flag for speed.
2	`apply_patches.sh` (revision check)	Reads `BASE_REVISION.txt` and compares it with `git rev-parse HEAD` in the Chromium checkout. Fails fast if there is a mismatch. Bypassable with `--skip-base-check`.
3	`apply_patches.sh`	Applies `*.patch` files first, then `*.diff` files, in alphabetical order from the `patches/` directory. Supports a custom patch directory via `--patch-dir`. Dry-run mode via `--check`.
4	`build_prism.sh` → `gn gen` + `ninja`	Generates the build system with `gn gen out/Default` and builds the `chrome` target via `ninja -C out/Default chrome`. Supports extra GN args via `--gn-args`.
5	GTest / pytest	Runs compiled C++ unit tests (`browser_tests`, `unit_tests`) and Python agent tests. Reports test results to CI.
6	Custom packaging	Bundles the Prism binary, homepage (`homepage-devspace/`), and Aura AI chat (`ai-chat/`) into a distributable archive (`.tar.gz` on Linux, `.zip` on Windows).
7	GitHub Releases API	Uploads the archive as a GitHub Release asset tagged with the Chromium base revision and a Prism version string.

7.5.3 Key CI/CD Properties

- **Reproducibility:** Every build is anchored to a specific Chromium revision via `BASE_REVISION.txt`, ensuring builds are deterministic and patches apply cleanly.
- **Incremental Builds:** The Ninja build system caches compilation results; only files modified by patches or upstream changes are recompiled, significantly reducing build time on iterative runs.
- **Patch Drift Detection:** If upstream Chromium changes a patched file, the `git apply` step will fail with a conflict report, immediately alerting contributors to regenerate affected patches.

- **Separation of Concerns:** Feature patches are kept in separate, numbered files (`0001-*`, `0002-*`, . . .), enabling individual features to be reviewed, reverted, or cherry-picked independently.
- **Minimal Repository Size:** The public Prism repository stores only patch files, scripts, and documentation – no Chromium source code – keeping repository size under 1 MB.

Chapter 8

Future Implementation

Prism AI Browser is a functional prototype demonstrating the feasibility of agentic, privacy-centric AI integration in a Chromium-based browser. Several enhancements and new features are identified for future development cycles.

8.1 Fully On-Device LLM via WebAssembly

Currently, the Aura AI Chat feature relies on external API calls to Gemini (Google) and DeepSeek (Hugging Face Inference API), which require internet connectivity and third-party API keys. A planned future enhancement is to replace or supplement these with a fully on-device LLM running via WebAssembly (WASM):

- Integrate **WebLLM** (web-llm.mlc.ai) or **Ollama** (local HTTP server) as a first-class LLM backend option alongside Gemini and DeepSeek.
- Package quantised model weights (e.g., Llama 3, Phi-3 Mini, Gemma 2B in GGUF format) alongside the browser distribution.
- Perform all inference in the browser process via the WebGPU-accelerated WebLLM runtime, eliminating any external API dependency for chat.
- Provide a model manager UI in the settings page for downloading, switching, and managing local model files.

8.2 Multi-Agent Coordination

The current AI agent is a single-agent system that handles one command at a time. Future work includes:

- A **planner agent** that decomposes complex, multi-part user goals into sub-tasks and delegates them to specialised sub-agents (e.g., a "Web Research Agent", a "Form-Filling Agent", a "Summarisation Agent").
- **Parallel tab execution**: Multiple agents working simultaneously across different browser tabs to complete parts of a task in parallel.
- An **agent memory bus** enabling agents to share intermediate results (e.g., data extracted from one page used as input for an action on another).

- A **task queue UI** in the browser showing running, queued, and completed agent tasks with the ability to pause, cancel, or replay them.

8.3 Enhanced Voice Interface

- Replace the current Web Speech API (which may use cloud STT) with a fully local, on-device speech recognition engine such as **Whisper** (OpenAI Whisper via WASM) or **Vosk**.
- Add wake-word detection (e.g., "Hey Prism") for hands-free activation without clicking the microphone button.
- Implement **Text-to-Speech (TTS)** output so Prism can read AI responses aloud, enabling fully eyes-free interaction.
- Support multi-language voice input beyond English.

8.4 Prism Extension Store and Plugin API

- Develop a lightweight **Prism Extension Store** hosting curated extensions optimised for AI-assisted browsing (AI summarisers, fact-checkers, automated form fillers).
- Expose a **Prism AI Plugin API**: a JavaScript interface allowing browser extensions to register new tools for the AI agent, define their schemas, and receive agent-dispatched function calls (similar in concept to OpenAI function calling / tool use).
- Enable third-party developers to build and distribute AI-powered browser extensions through the Prism plugin ecosystem.

8.5 Advanced Image Generation Features

- Add support for additional image generation models beyond FLUX.2 Klein, including **Stable Diffusion XL** (local, via WASM) and **Google Imagen** (API).
- Implement **image editing** commands: "Make the background blue", "Remove the person from this image".
- Provide an **in-page image generation overlay** accessible directly from any webpage via right-click context menu.
- Add a **prompt history** panel in Aura AI Chat to quickly re-run and iterate on previous image generation prompts.

8.6 Homepage DevSpace Enhancements

- **Custom widget system**: Allow users to add/remove/reorder widgets on the homepage (RSS feeds, GitHub activity, weather, stock ticker, Pomodoro timer).
- **Theme engine**: A visual theme builder with live preview for selecting font, colour palette, background, and animation style; themes exportable and shareable as JSON files.
- **Deeper browser integration**: Link Dev Space tools (Color Gen, Prompt Synthesizer) directly to Aura AI Chat for AI-assisted colour palette generation and prompt refinement from within the homepage.
- **Offline-first**: Service Worker caching so the homepage fully loads with no internet connection.

8.7 Privacy and Security Hardening

- **AI Content Policy Engine**: A locally-running lightweight classifier that flags potentially sensitive content before it is passed to any LLM (including local ones) and prompts the user for confirmation.
- **CDP Access Control**: Restrict CDP access with an authentication token so that only the authorised Prism AI agent server can connect, preventing rogue local processes from exploiting the always-on CDP port.
- **Encrypted local memory**: Encrypt the `localStorage`-based AI memory using the OS keychain, so memory data cannot be read by other applications.
- **Fine-grained permission model**: Allow users to restrict which websites the AI agent is allowed to interact with (allowlist/blocklist per domain).

8.8 Cross-Platform and Mobile Support

- Extend the patch set and build scripts to fully support **macOS** (ARM and x86-64) and **Windows** as first-class build targets alongside Linux.
- Explore a **Prism for Android** build based on Chromium's Android target, with a mobile-adapted Aura AI Chat that uses a smaller on-device model (Gemma 2B, Phi-3 Mini).
- Provide pre-built binary distributions via GitHub Releases so that non-developer users can install Prism without needing to compile Chromium from source.

8.9 Automated Patch Rebase Tooling

- Build a **rebase assistant** GitHub Action that, when upstream Chromium is updated, automatically attempts to apply the Prism patches to the new revision, reports any conflicts, and opens a pull request with the rebase result.
- Develop a **conflict resolution guide** that analyses which upstream Chromium files were changed in the new revision and cross-references them with the Prism patch set to predict rebase difficulty before attempting the rebase.

Project Snapshots

The following screenshots illustrate the key features and interfaces of the Prism AI Browser as developed and tested during the project.

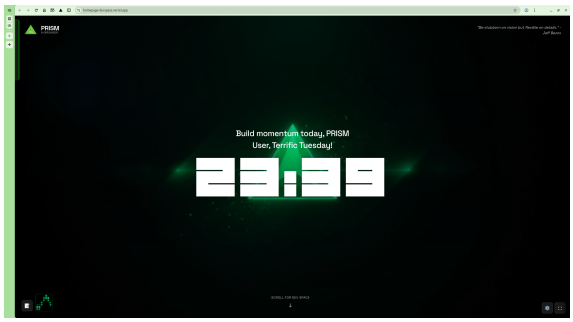

Figure 8.1: Screenshot 1: Prism Browser — Home Screen

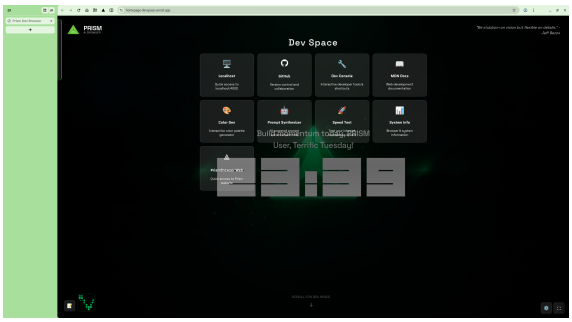

Figure 8.2: Screenshot 2: DevSpace Homepage

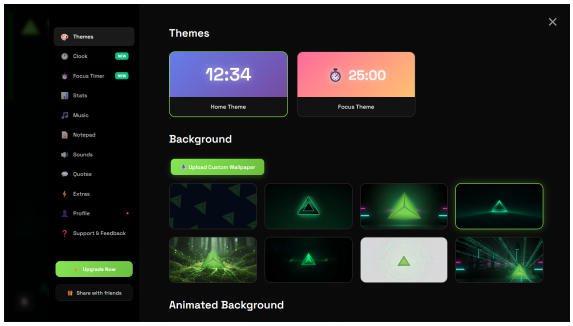

Figure 8.3: Screenshot 3: DevSpace Settings Panel

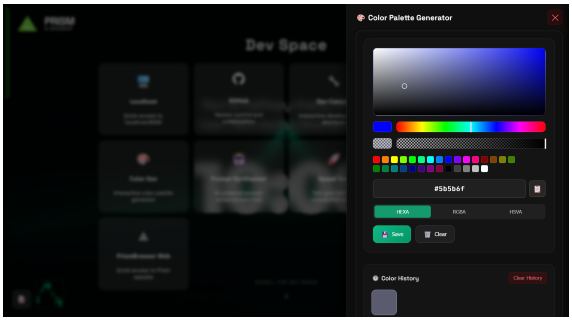

Figure 8.4: Screenshot 4: Colour Generator Tool

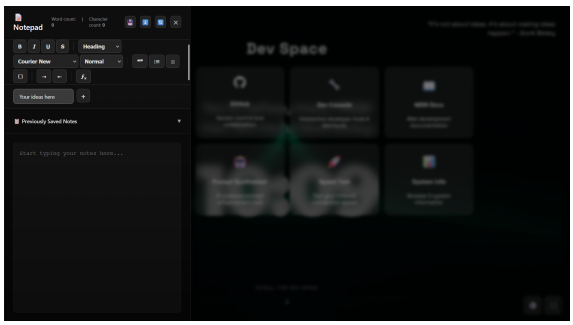

Figure 8.5: Screenshot 5: Notepad Tool

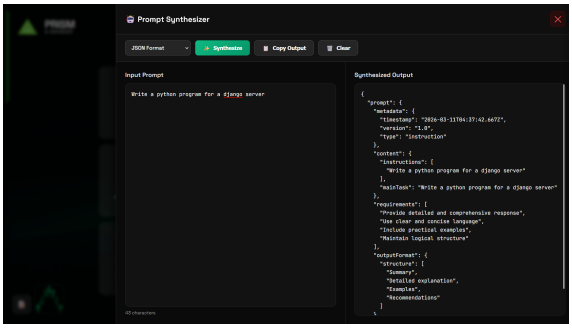

Figure 8.6: Screenshot 6: Prompt Synthesizer Tool

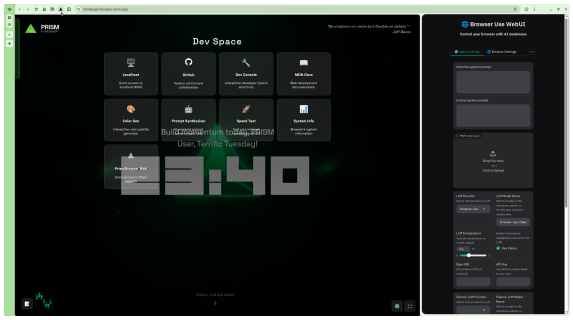

Figure 8.7: Screenshot 7: Agentic Task UI

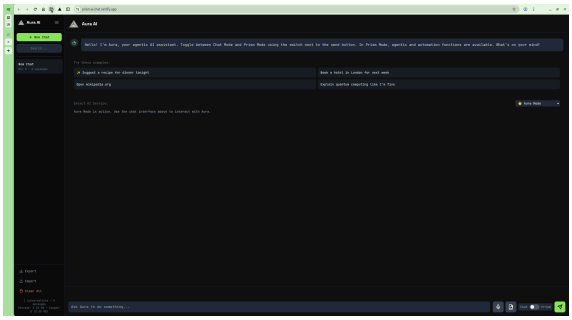

Figure 8.8: Screenshot 8: Aura AI Chat Interface

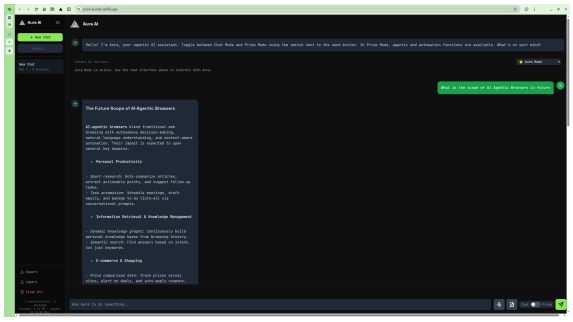

Figure 8.9: Screenshot 9: Chat Response Output

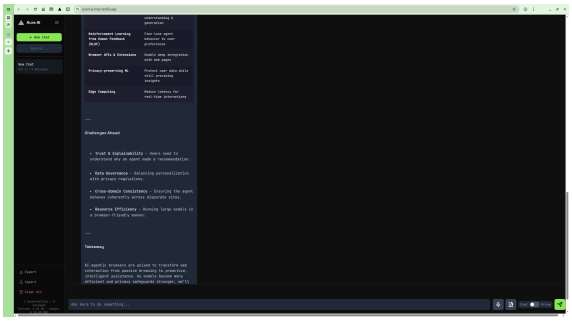

Figure 8.10: Screenshot 10: Chat Response Output (Continued)

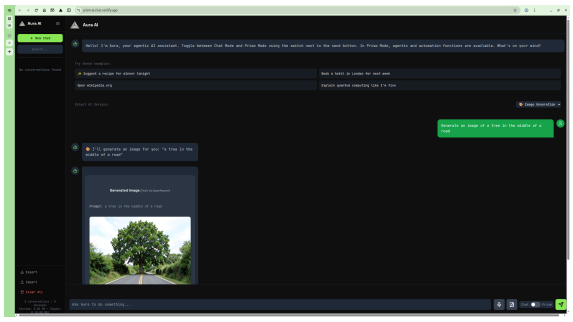

Figure 8.11: Screenshot 11: AI Image Generation

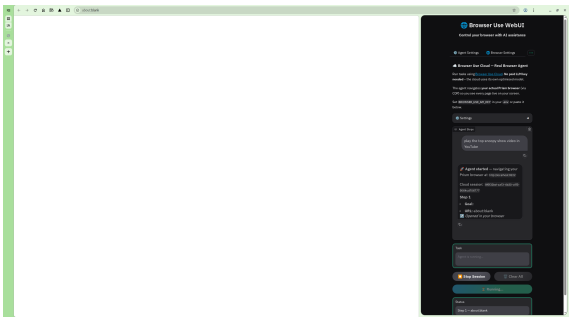

Figure 8.12: Screenshot 12: Agentic Task in Progress

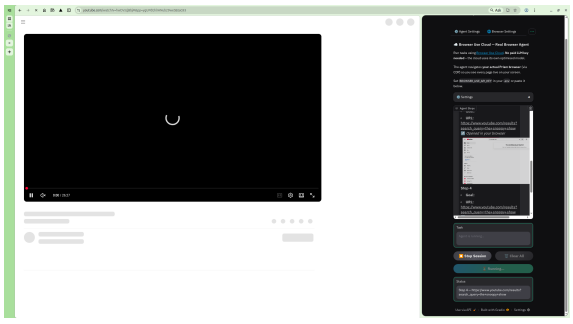

Figure 8.13: Screenshot 13: Agentic Task Output

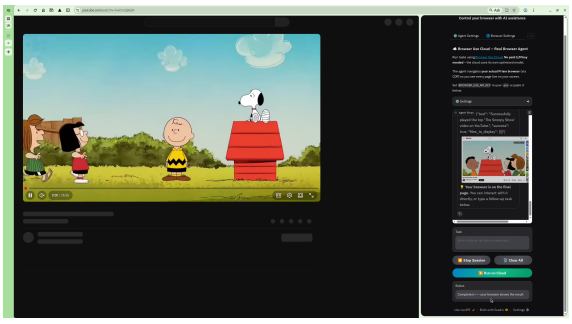

Figure 8.14: Screenshot 14: Agentic Task Completed Successfully

Chapter 9

Conclusion

9.1 Summary of Work

This project set out to build **Prism AI Browser** – an intelligent, agentic, and privacy-centric web browser – by extending the open-source Chromium browser with tightly integrated AI capabilities. Over the course of the project, the team designed, implemented, tested, and documented a complete system comprising three interlocking components:

1. **Prism Chromium Build** – A patch-based customisation of the Chromium browser that renames the product to "Prism", adds a custom `PrismAIButton` in the main toolbar, and guarantees the Chrome DevTools Protocol (CDP) is always active on port 9222.

2. **Aura AI Chat** – A full-featured conversational AI web application supporting multi-model chat (Gemini Pro, DeepSeek V3.2), AI image generation (FLUX.2 Klein via OpenRouter), natural language browser task execution, persistent conversation history, and a privacy-respecting local memory system.

3. **Prism Homepage DevSpace** – A custom new-tab homepage for developer users, offering a live clock, motivational quotes, animated backgrounds, RAM monitoring, and a suite of embedded developer tools (Color Generator, Prompt Synthesizer, Speed Test, Notepad, Focus Mode).

9.2 Key Achievements

- **Patch-first architecture:** All Prism-specific modifications to Chromium are captured as compact, reproducible Git patch files totalling under 1 MB, allowing any contributor to rebuild Prism from a standard Chromium checkout in a single command (`build_prism.sh`).

- **CDP-driven automation:** By guaranteeing CDP availability at browser startup, the AI agent can programmatically control every aspect of the browser – navigation, DOM inspection, input simulation, JavaScript execution – without requiring any browser extension or external driver.

- **Agentic execution:** The `TaskExecutor` module demonstrates true agentic behaviour, understanding natural language commands and executing multi-step browser actions (open, search, interact) autonomously with user confirmation checkpoints for safety.

- **Privacy by design:** All AI inference flows (chat, task execution) operate over the loopback interface only. No user data, page content, or browsing history is transmitted to external servers as part of the core AI pipeline. Cloud API calls (Gemini, DeepSeek, OpenRouter for images) are opt-in, configured by the user, and transport only the explicit query.
- **Multi-model flexibility:** The chat interface supports seamless switching between AI models and includes exponential backoff retry logic with real-time user feedback, ensuring a robust experience even under API instability.
- **Developer-first UX:** The DevSpace homepage and the embedded tooling (Prompt Synthesizer, Color Gen, Speed Test) demonstrate Prism's positioning not just as a browser but as a developer environment.

9.3 Challenges Overcome

- **Chromium build complexity:** The Chromium source tree is one of the largest open-source codebases in existence. Setting up `depot_tools`, managing `gclient` dependencies, and understanding the GN build system required significant upfront effort.
- **Patch maintenance:** Upstream Chromium evolves rapidly. Ensuring patches applied cleanly to a pinned revision and understanding how to regenerate patches after upstream changes was a key engineering challenge.
- **CDP integration:** The Chrome DevTools Protocol has extensive documentation but subtle behaviours – particularly around tab lifecycle, session management, and asynchronous event ordering – that required careful handling in the Python agent.
- **LLM reliability:** Cloud LLM APIs (particularly Hugging Face Inference) can be slow or return 503 errors under load. Implementing robust exponential backoff retry logic with live user feedback was an important quality-of-life improvement.
- **CORS and local serving:** Serving the homepage and chat app locally required careful configuration to avoid CORS errors, leading to the adoption of a Python/Node.js local HTTP server as the standard serving mechanism.

9.4 Learning Outcomes

The team gained practical experience in the following areas:

- Large-scale C++ software development and modification within a real-world production codebase (Chromium).
- Cross-platform build systems (GN, Ninja) and patch-based software distribution workflows.
- Browser internals: the Views UI toolkit, toolbar architecture, CDP protocol, and browser process lifecycle.
- Asynchronous Python programming with WebSocket clients and HTTP servers.
- Frontend development with Tailwind CSS, JavaScript ES6+, and Web APIs (Speech, Performance, localStorage).
- API integration patterns: REST, WebSocket, exponential backoff retry, and API key management.
- AI application development: prompt engineering, multi-model routing, image generation integration, and local memory systems.
- Software documentation: UML diagrams (Use Case, Class, Sequence, State Chart, Flowchart), requirements engineering (SRS format), and academic technical writing.

9.5 Conclusion

Prism AI Browser demonstrates that it is both feasible and practical to deeply integrate agentic AI capabilities into a modern web browser without sacrificing privacy. By combining a lean patch set atop Chromium, a CDP-driven Python AI agent, and a polished AI chat and developer homepage, the project delivers a coherent vision of what the next generation of intelligent browsers could look like.

The foundation laid by this project – the reproducible build system, the CDP automation backbone, the multi-model AI chat, and the developer-focused homepage – provides a solid platform for the rich set of future enhancements described in Chapter 8. With planned improvements such as fully on-device LLM inference, multi-agent coordination, an extension plugin API, and broader platform support, Prism has the potential to grow from a research prototype into a genuinely useful product.

The project successfully meets all its primary objectives: the browser is buildable, the AI features are functional and privacy-respecting, and the overall system architecture is maintainable and extensible. The team concludes that the integration of agentic AI into the browser layer represents a meaningful direction for future browser development, and Prism AI Browser is a concrete step toward that future.

References

[1] The Chromium Projects, "Chromium Source Code Repository," *Google Git*, 2024. https://chromium.googlesource.com/chromium/src

[2] Google Chrome DevTools Team, "Chrome DevTools Protocol (CDP)," *Chrome DevTools Protocol Viewer*, 2024. https://chromedevtools.github.io/devtools-protocol/

[3] Google LLC, "Chromium depot_tools Documentation," *Chromium Build Tools*, 2024. https://commondatastorage.googleapis.com/chrome-infra-docs/flat/depot_tools/docs/html/depot_tools_tutorial.html

[4] The Chromium Projects, "GN Build System Reference," *Chromium Docs*, 2024. https://gn.googlesource.com/gn/+/main/docs/reference.md

[5] Ninja Build System, "Ninja: a small build system with a focus on speed," 2024. https://ninja-build.org/

[6] Google DeepMind, "Gemini API Documentation," *Google AI for Developers*, 2024. https://ai.google.dev/docs

[7] DeepSeek AI, "DeepSeek-V3 Technical Report," *Hugging Face*, 2024. https://huggingface.co/deepseek-ai/DeepSeek-V3

[8] OpenRouter, "OpenRouter API – Unified AI Model Gateway," 2024. https://openrouter.ai/docs

[9] Black Forest Labs, "FLUX.1 Image Generation Models," *Hugging Face*, 2024. https://huggingface.co/black-forest-labs

[10] G. Ggerganov, "llama.cpp – Local LLM Inference in C/C++," *GitHub*, 2024. https://github.com/ggerganov/llama.cpp

[11] Mozilla Developer Network, "Web Speech API," *MDN Web Docs*, 2024. https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API

[12] Mozilla Developer Network, "Performance.memory API," *MDN Web Docs*, 2024. https://developer.mozilla.org/en-US/docs/Web/API/Performance/memory

[13] Mozilla Developer Network, "Window.localStorage," *MDN Web Docs*, 2024. https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage

[14] Tailwind CSS, "Tailwind CSS Documentation," 2024.
https://tailwindcss.com/docs

[15] OpenAI, "GPT API Reference," *OpenAI Platform Docs*, 2024.
https://platform.openai.com/docs/

[16] Python Software Foundation, "asyncio – Asynchronous I/O," *Python 3 Documentation*, 2024.
https://docs.python.org/3/library/asyncio.html

[17] A. Padmanabha Iyer, "Building Agentic AI Systems," *arXiv preprint*, 2023.
https://arxiv.org/abs/2308.11432

[18] Browser Use, "browser-use: Make Websites Accessible for AI Agents," *GitHub*, 2024.
https://github.com/browser-use/browser-use